

Sprawozdanie LAB 5-7

Szymon Krzysztofik

LAB 5

Wybrano aplikację opartą o repozytorium Marked.js. Zweryfikowano licencję oprogramowania, która potwierdza możliwość swobodnego obrotu kodem na potrzeby tego zadania. Utworzono forka repozytorium na potrzeby implementacji procesu budowania.

Środowisko Jenkins zostało postawione zgodnie z instrukcjami. Uruchomiono kontener z Jenkinsem oraz obraz Dockera eksponujący środowisko DIND.

```
docker run --name jenkins-docker --rm --detach \
  --privileged --network jenkins --network-alias docker \
  --env DOCKER_TLS_CERTDIR=/certs \
  --volume jenkins-docker-certs:/certs/client \
  --volume jenkins-data:/var/jenkins_home \
  docker:dind
```

```
docker run --name jenkins-server --rm --detach \
  --network jenkins --env DOCKER_HOST=tcp://docker:2376 \
  --env DOCKER_CERT_PATH=/certs/client --env DOCKER_TLS_VERIFY=1 \
  --volume jenkins-data:/var/jenkins_home \
  --volume jenkins-docker-certs:/certs/client:ro \
  --publish 8080:8080 --publish 50000:50000 \
  moj-jenkins-docker
```

Uruchomiono interfejs Blue Ocean i poprawnie skonfigurowano Jenkinsa. Zadbano o archiwizację logów oraz ich zabezpieczenie.

Wykonano zadania wstępne potwierdzające działanie środowiska:

Utworzono projekt wyświetlający polecenie uname.

✓ Test-Unix 1

Pipeline

Changes

Tests

Artifacts

⚙️

🔗

Logout

✕

ae

Branch: — 🕒 <1s No changes

Commit: — 🕒 4 hours ago Started by user Szymon Krzysztofik

Logs 🔗 📄

```
1 Started by user Szymon Krzysztofik
2 Running as SYSTEM
3 Building in workspace /var/jenkins_home/workspace/Test-Unix
4 [Test-Unix] $ /bin/sh -xe /tmp/jenkins13519827601586138920.sh
5 + uname -a
6 Linux 91f9ff318159 6.18.13-200.fc43.x86_64 #1 SMP PREEMPT_DYNAMIC Thu Feb 19 19:54:01 UTC 2026 x86_64
  GNU/Linux
7 Finished: SUCCESS
```

Utworzono projekt zwracający błąd przy nieparzystej godzinie.

✕ Test-Godzina < 8

Pipeline

Changes

Tests

Artifacts

⚙️

🔗

Logout

✕

ae

Branch: — 🕒 <1s No changes

Commit: — 🕒 a few seconds ago Started by user Szymon Krzysztofik

Logs 🔗 📄

```
1 Started by user Szymon Krzysztofik
2 Running as SYSTEM
3 Building in workspace /var/jenkins_home/workspace/Test-Godzina
4 [Test-Godzina] $ /bin/bash /tmp/jenkins4515490057127082713.sh
5 Aktualna godzina to: 11
6 Błąd! Godzina jest nieparzysta, przerywam zadanie!
7 Build step 'Execute shell' marked build as failure
8 Finished: FAILURE
```

Pobrano obraz systemowy przy użyciu polecenia docker pull ubuntu.

✓ Test-Docker-Pull < 4

Pipeline

Changes

Tests

Artifacts

⚙️

🔗

Logout

✕

ae

Branch: — 🕒 6s No changes

Commit: — 🕒 8 minutes ago Started by user Szymon Krzysztofik

Logs 🔗 📄

```
1 Started by user Szymon Krzysztofik
2 Running as SYSTEM
3 Building in workspace /var/jenkins_home/workspace/Test-Docker-Pull
4 [Test-Docker-Pull] $ /bin/sh -xe /tmp/jenkins13816164785132717055.sh
5 + docker pull ubuntu:22.04
6 22.04: Pulling from library/ubuntu
7 96c832531c38: Pulling fs layer
8 2190f99273e0: Download complete
9 96c832531c38: Download complete
10 96c832531c38: Pull complete
11 Digest: sha256:ce4a593b4e323dcc3dd728e397e0a866a1bf516a1b7c31d6aa06991baec4f2e0
12 Status: Downloaded newer image for ubuntu:22.04
13 docker.io/library/ubuntu:22.04
14 Finished: SUCCESS
```

Projekt procesu CI/CD
Działający serwer Jenkins powiązany z demonem Dockera (DIND) oraz dostęp do repozytorium GitHub.

✓ Lab5-Pipeline 1

Pipeline

Changes

Tests

Artifacts

Logout

Branch: —

1m 46s

No changes

Commit: —

14 minutes ago

Started by user Szymon Krzysztofik



Test - 25s [Restart Test](#) [Link](#) [Download](#)

✓	> Budowanie obrazu testowego i uruchamianie testów... — Print Message	<1s
✓	> docker build -t marked-test:latest -f Dockerfile.test . — Shell Script	1s
✓	> docker run --rm marked-test:latest — Shell Script	24s

Jenkins

Pipelines

Administration

Log out

Lab5-Pipeline ☆ ⚙

Activity

Branches

Pull Requests

Run

Disable

STATUS	RUN	COMMIT	MESSAGE	DURATION	COMPLETED	
✓	3	—	Started by user Szymon Krzysztofik	34s	a few seconds ago	↺
✓	2	—	Started by user Szymon Krzysztofik	1m 32s	3 minutes ago	↺
✓	1	—	Started by user Szymon Krzysztofik	1m 46s	7 hours ago	↺

Diagram aktywności: Pokazuje kolejne etapy procesu rurociągu (collect, build, test, report/publish)

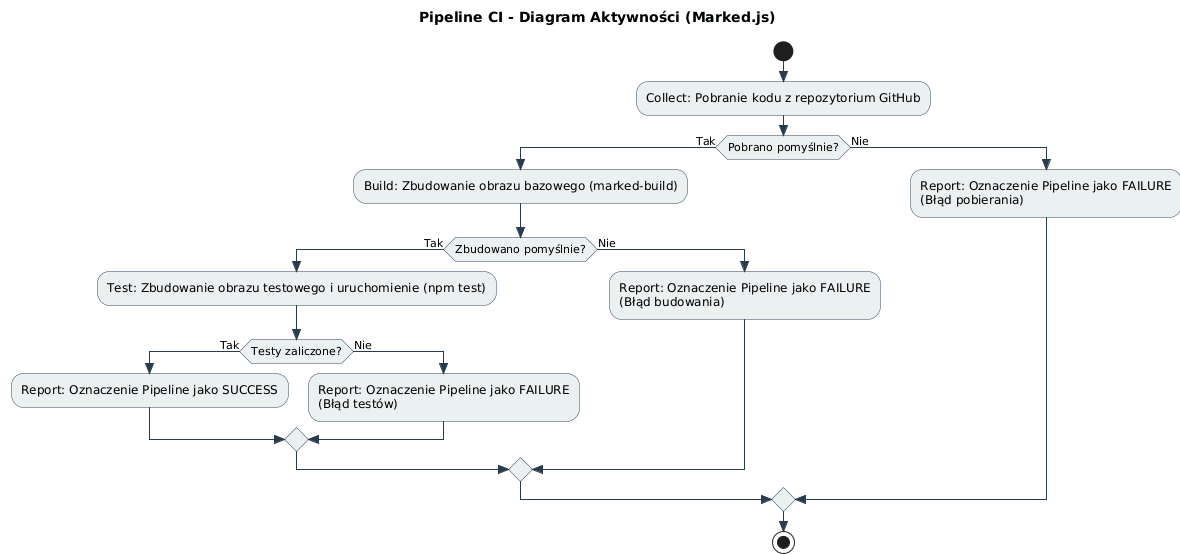
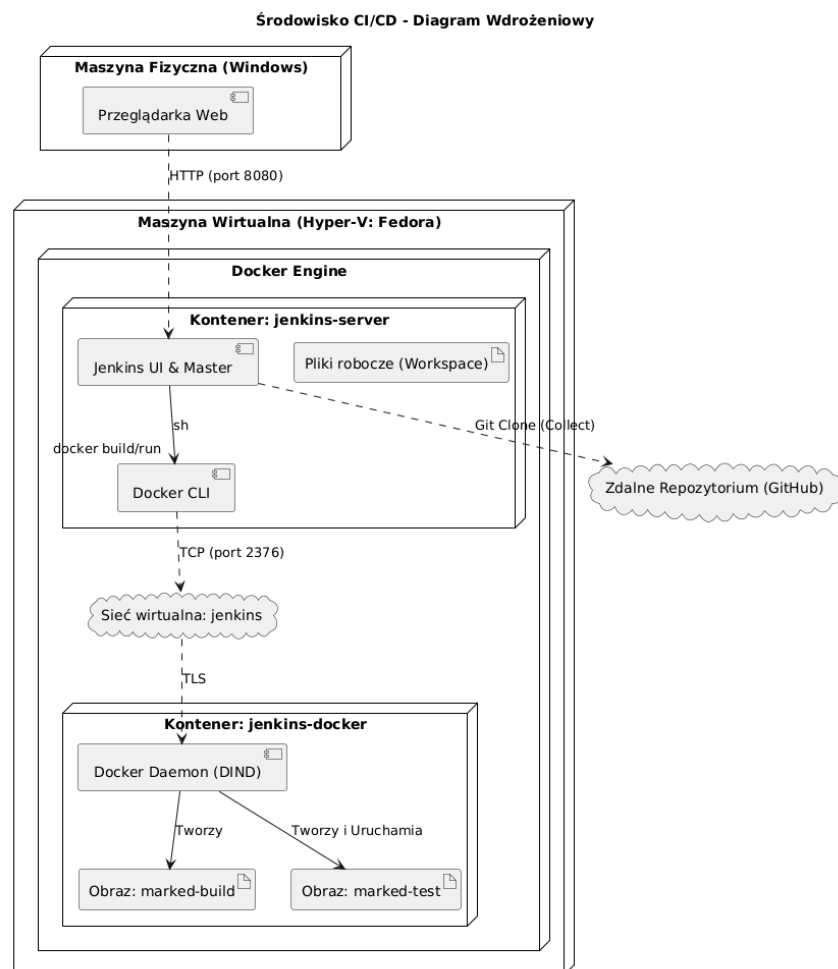


Diagram wdrożeniowy: Opisuje relacje między maszynami wirtualnymi, kontenerami, zasobami zewnętrznymi i artefaktami.



LAB 6

Zdefiniowano obiekt pipeline realizujący pełną ścieżkę krytyczną

Clone: Kod pobierany jest prosto z repozytorium.

Build: Proces wykorzystuje zdefiniowane pliki Dockerfile i buduje obraz wewnątrz dedykowanego kontenera bazowego zawierającego wymagane dependencje (użyto konkretnego tagu z wersją obrazu Node, a nie :latest).

Test: Dołączone do programu testy uruchamiają się w kolejnym, dedykowanym kontenerze opartym o środowisko budujące. Logi z procesu udostępniono do analizy.

Deploy: Zdefiniowano lekki kontener wdrożeniowy (sandbox) pozbawiony zależności z etapu builda. Wdrożenie na tym etapie następuje poprzez start tego kontenera i wykonanie smoke testu.

Publish: Postacią redystrybucyjną artefaktu jest archiwum [.tar.gz](#).

The screenshot displays the Jenkins Pipeline interface. At the top, a green bar indicates the pipeline is successful, labeled 'Pipeline < 10'. Below this, a summary bar shows the branch, commit, and execution details. The main part of the interface shows a pipeline graph with stages: Start, Build Builder, Test, Deploy (Smoke Test), Publish Artifact, and End. The 'Publish Artifact' stage is currently selected and highlighted with a blue circle. Below the graph, the 'Publish Artifact - 1s' stage is expanded, showing a list of steps with their durations. The steps include: 'Wydobywanie plików z kontenera...', 'docker create --name extract-container marked-deploy:latest', 'docker cp extract-container:/app ./marked-production', 'docker rm extract-container', 'Pakowanie do formatu .tar.gz...', 'tar -czvf marked-artifact-build-\${BUILD_NUMBER}.tar.gz ./marked-production', 'Publikowanie artefaktu w Jenkinsie...', and 'Archive the artifacts'.

Step	Duration
> Wydobywanie plików z kontenera... — Print Message	<1s
> docker create --name extract-container marked-deploy:latest — Shell Script	<1s
> docker cp extract-container:/app ./marked-production — Shell Script	<1s
> docker rm extract-container — Shell Script	<1s
> Pakowanie do formatu .tar.gz... — Print Message	<1s
> tar -czvf marked-artifact-build-\${BUILD_NUMBER}.tar.gz ./marked-production — Shell Script	<1s
> Publikowanie artefaktu w Jenkinsie... — Print Message	<1s
> Archive the artifacts	<1s

Publikacja Artefaktu i Weryfikacja UML

Dostępność artefaktu: Wersjonowany artefakt `.tar.gz` został pomyślnie wyekstrahowany z obrazu docelowego, spakowany i załączony jako pobieralny wynik zlecenia w Jenkinsie

✓ Pipeline < 10

PipelineChangesTestsArtifacts🔄⚙️📄Logout👤

Branch: —

🕒 1h 3m 10s

Changes by Szymon Krzystofik

Commit: —

🕒 5 minutes ago

Started by user Szymon Krzystofik

NAME	SIZE	
pipeline.log	-	📄 📄
marked-artifact-build-10.tar.gz	167.1 KB	📄 📄

Download All

Zdefiniowano niezbędne obrazy infrastruktury budującej:

Dockerfile:

```
FROM node:20
```

```
WORKDIR /app
```

```
RUN git clone https://github.com/markedjs/marked.git .
```

```
RUN npm install
```

```
RUN npm run build
```

Dockerfile.test:

```
FROM marked-build:latest
```

```
CMD ["npm", "test"]
```

Dockerfile.deploy:

```
FROM marked-build:latest AS builder
```

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY --from=builder /app/package.json ./
```

```
COPY --from=builder /app/lib ./lib
```

```
COPY --from=builder /app/bin ./bin
```

```
RUN npm install --omit=dev
```

```
CMD ["node", "bin/marked.js", "--version"]
```

LAB 7

Stworzony pipeline nie żyje w ustawieniach Jenkinsa, ale został umieszczony w repozytorium kodu jako plik Jenkinsfile, spełniając wszystkie kryteria weryfikacyjne

Jenkinsfile:

```
pipeline {
    agent any

    stages {
        stage('Build Builder') {
            steps {
                dir('ITE/GCL3/SK419875/markedjs-projekt') {
                    sh 'docker build -t marked-build:latest -f
Dockerfile .'
                }
            }
        }

        stage('Test') {
            steps {
                dir('ITE/GCL3/SK419875/markedjs-projekt') {
                    sh 'docker build -t marked-test:latest -f
Dockerfile.test .'
                    sh 'docker run --rm marked-test:latest'
                }
            }
        }

        stage('Deploy (Smoke Test)') {
            steps {
                dir('ITE/GCL3/SK419875/markedjs-projekt') {
                    echo 'Budowanie lekkiego kontenera
produkcyjnego...'
                    sh 'docker build -t marked-deploy:latest -f
Dockerfile.deploy .'
                    echo 'Uruchamianie Smoke Testu...'
                    sh 'docker run --rm marked-deploy:latest'
                }
            }
        }

        stage('Publish Artifact') {
            steps {
                dir('ITE/GCL3/SK419875/markedjs-projekt') {
                    echo 'Wydobywanie plikow z kontenera...'
                    sh 'docker create --name extract-container
marked-deploy:latest'
```

```

        sh 'docker cp extract-container:/app
./marked-production'
        sh 'docker rm extract-container'

        echo 'Pakowanie do formatu .tar.gz...'
        sh 'tar -czvf
marked-artifact-build-${BUILD_NUMBER}.tar.gz ./marked-production'

        echo 'Publikowanie artefaktu w Jenkinsie...'
        archiveArtifacts artifacts: '*.tar.gz',
fingerprint: true
    }
}
}

post {
    always {
        echo 'Czyszczenie srodowiska (Workspace) po
zakonczeniu rurociagu...'
        deleteDir() // Usuwa pliki z katalogu roboczego
Jenkinsa
    }
}
}

```

Przepis jest dostarczany ze zdefiniowanego repozytorium SCM.

Rurociąg wykonuje sprzątanie po swojej pracy (`deleteDir()`), co gwarantuje, że każde kolejne ponowne uruchomienie opiera się na najświeższym kodzie (nie polega na cache'u środowiska pracy Jenkinsa).

Etapy Build, Test, Deploy i Publish są logicznie oddzielone i posiadają inne przeznaczenie docelowe kontenerów.

"Definition of Done"

Zaimplementowany proces w 100% zaspokaja definicję ukończenia

Docelowym wynikiem "na końcu rurociagu" jest artefakt gotowy do wdrożenia.

Pobieralny artefakt zawiera odizolowaną, gotową do uruchomienia postać programu. Nie wymaga on budowania od zera (wymaga jedynie środowiska uruchomieniowego, np. Node.js).